

Basic Data Analysis with SPARK

Let an LLM write and run Python on a CSV - generate plots and statistics from plain-English requests (slides 24-25).

Activity is designed by Su Nguyen; Detailed guide is drafted by Claude Cowork.

Before you start

You need an API key set in Settings (gear, top-right). The activity uses Google Gemini (gemini-2.5-flash); paste your Gemini key (the same one from Week 5) and Save Settings. If Gemini returns a "high demand" error, switch the AI Provider on the Python Analyzer node to another model (e.g. GPT) and re-run - the flow works the same way. The data file (mpg.csv) is in the Week 6 > data > csv folder.

Tool	SPARK - Studio for Prototyping Agent Reasoning & Knowledge
Model	Google Gemini (gemini-2.5-flash) - or GPT if Gemini is busy
Input	mpg.csv - the classic auto-mpg dataset (~400 cars)
What you build	Car Data (CSV) -> My Request -> Python Analyzer -> result (code + text + plot)
Outcome	A scatter plot and average statistic, then three charts grouped by country of origin

Two companion screen recordings (Week6_SPARK_DataAnalysis_Base.gif, Week6_SPARK_DataAnalysis_Charts.gif) show every click in sequence.

Step 1 - Load the CSV Analysis with Python example (Slide 24)

This flow is different from the others: instead of just answering in text, the LLM writes Python code, runs it in the browser (via Pyodide), and returns whatever the code produces - numbers and charts included.

1. On the Gallery tab, find the CSV Analysis with Python card ("Uses the Data Analyzer to run AI-generated Python code on a CSV file to create a plot and calculate statistics").
2. Click Load Example, then Confirm to clear the canvas.
3. The Canvas opens with four nodes: Car Data (the CSV), My Request (your instruction), Python Analyzer (the LLM that writes/runs code), and Display Result.

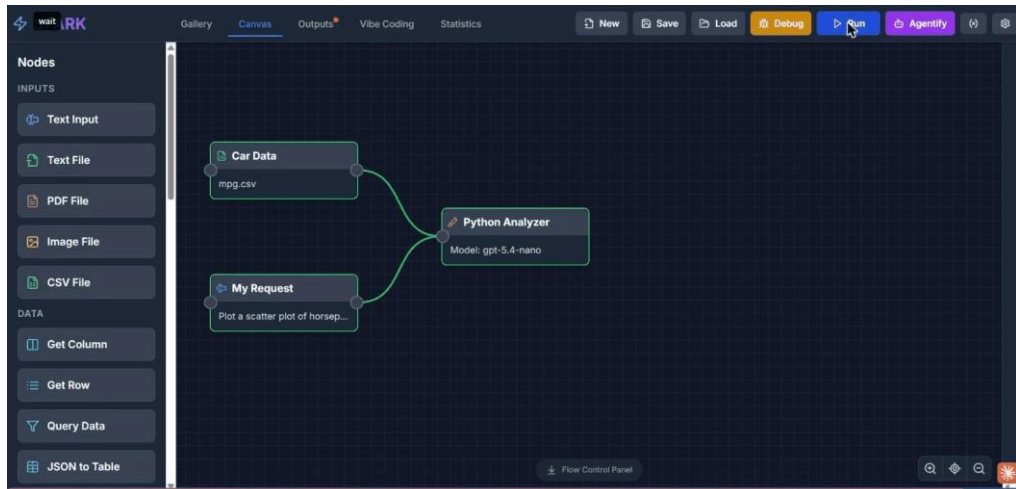


Figure 1 - The flow: Car Data (mpg.csv) + My Request -> Python Analyzer -> result. The Python Analyzer writes and runs the code.

Step 2 - Set the model and upload mpg.csv (Slide 24)

4. Click the Python Analyzer node. In Properties, set AI Provider to Google Gemini (Model: gemini-2.5-flash).
5. Click the Car Data node, then Upload CSV File and choose mpg.csv. Confirm the node now shows mpg.csv (not the default cars.csv) and that First row is header is ticked.
6. Leave the My Request node as the existing prompt below.

```
Plot a scatter plot of horsepower vs weight. Also, print the average horsepower.
```

Tip: if Gemini reports it is "experiencing high demand", switch the Python Analyzer's AI Provider to a GPT model and run again - everything else stays the same.

Step 3 - Run and read the result (Slide 24)

Click Run. When it finishes, click the Python Analyzer node to open the Analysis Result dialog. It has three panes:

- Generated Python Code - the pandas/matplotlib code the model wrote (it even handles missing values).
- Text Output - here the average horsepower, 104.47.
- Generated Plot - the scatter plot of horsepower vs weight for all ~400 cars.

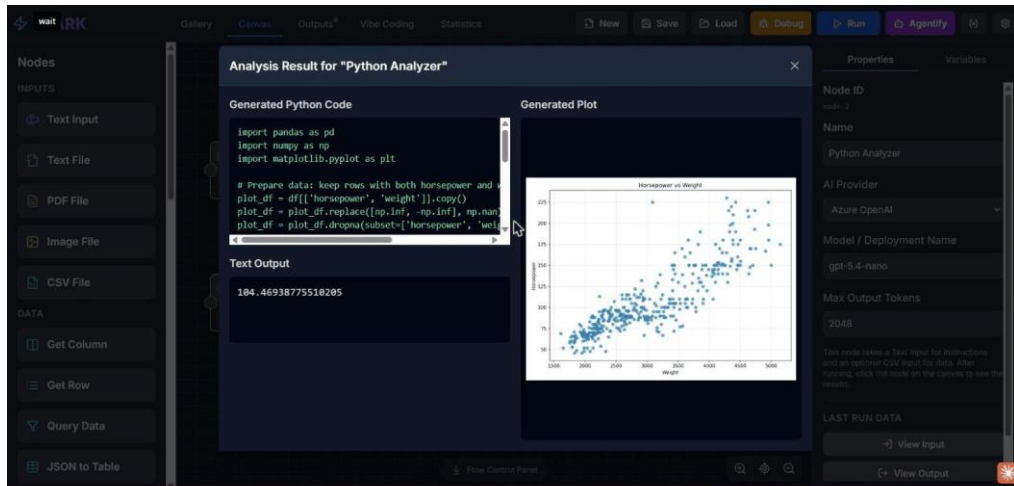


Figure 2 - Analysis Result: generated code, the average horsepower (104.47), and the horsepower-vs-weight scatter plot.

Tip: always skim the Generated Python Code. It shows exactly how the number and chart were produced - useful for trusting (or debugging) the result.

Step 4 - Edit the prompt to generate more charts (Slide 25)

The whole analysis is driven by the My Request text, so richer questions produce richer output. Click the My Request node and replace its text with a request for three charts grouped by country of origin (the 'origin' column in the dataset: 1, 2, 3).

Using the 'origin' column as the country of origin, create three charts: (1) a histogram showing the distribution of MPG by country of origin, (2) a box plot comparing MPG across the countries of origin, and (3) a scatter plot of MPG vs weight, colored by country of origin.

Tip: be explicit about the column name ('origin') and the chart types you want. The clearer the request, the closer the result matches what you had in mind.

Step 5 - Run and view the three charts (Slide 25)

Click Run, then open the Python Analyzer result again. The Generated Plot now contains all three charts side by side. (There is no Text Output this time - the request only asked for charts.)

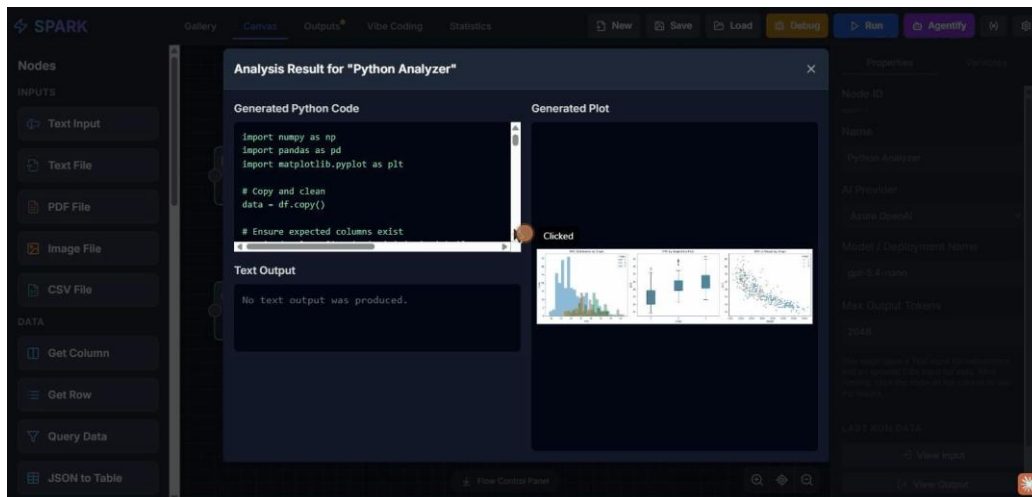


Figure 3 - The Analysis Result with three charts generated from a single plain-English request.

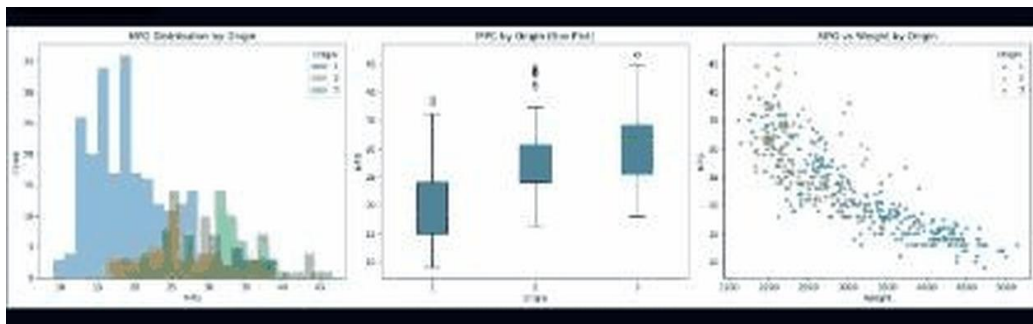


Figure 4 - Close-up of the three charts: MPG distribution by origin (histogram), MPG by origin (box plot), and MPG vs weight coloured by origin (scatter).

Together these tell a clear story: origin 1 (US) cars cluster at lower MPG and higher weight, while origins 2 and 3 (Europe and Japan) tend to be lighter and more fuel-efficient - exactly what the histogram, box plot, and coloured scatter each show from a different angle.

Good to know

- The Python Analyzer runs real Python (pandas, numpy, matplotlib) in your browser - the LLM only writes the code; Pyodide executes it.
- Your plain-English request is the whole interface: change the wording to change the analysis, the statistics, or the chart types.
- Name the exact columns you mean ('origin', 'horsepower', 'weight') so the model does not have to guess.
- If Gemini is overloaded, switch the AI Provider to a GPT model and re-run; the rest of the flow is unchanged.
- If the Car Data node still shows cars.csv after uploading, re-open the node and upload again - the canvas label should change to mpg.csv.